计算机体系结构

2. 性能评测

李建华
计算机与信息学院

The contents of the slides are adapted from CA course of wisc, princeton, mit, berkeley, edinburg, and eth. The uses of the slides of this course are for educational purposes only and should be used only in conjunction with the textbook. Derivatives of the slides must acknowledge the copyright notices of this and the originals.

性能分析

- 下面哪一种飞机的性能最好?

Airplane	Passengers	Range (mi)	Speed (mph)
Boeing 737-100	101	630	598
Boeing 747	470	4150	610
BAC/Sud Concorde	132	4000	1350
Douglas DC-8-50	146	8720	544

- 目标问题可能涉及多项性能指标，很难回答。
- Concorde与Boeing 747相比，快多少?
- Boeing 747与Douglas DC-8-50相比，大多少?

性能分析

- 再考虑这个问题：假设你要够买1台计算机，现在有若干台计算机可选，你如何选择?
  - 你当初购买笔记本的时候，是怎么考虑的?
- 要回答该问题，并不简单：
  - 科学仿真 – 浮点性能
  - 程序设计 – 整型性能
  - 数据库应用 – 存储性能 和 I/O性能
  - AI应用 – 计算并行性和存储带宽
- 涉及计算机的性能指标也有很多，购买的时候需要考虑**目标应用**及**影响应用性能的核心指标。**

计算机的性能

- 如果你想**购买**速度最快的计算机，需要先考虑你用它来**做什么**？
  - 游戏应用？程序开发？还是运行在线数据库？
  - 换句话说，运行的目标程序 (workloads) 很重要
- 除此之外，从**专业角度**来说，还要掌握正确的**性能评测和分析方法。**
- 如果你想**设计**速度最快的计算机，需要先考虑消费者愿意**为什么而付费**？
  - 从这个角度来说，运行的目标应用同样很重要；
  - 此外，从**生产者**角度，开销 (cost) 也非常重要。

本讲提纲

- 时间与性能
- Iron Law * – 时间
- 基准测试
- Amdahl’s Law

* Iron Law一词，来源于法律。 英 / ˈaɪən / 美 / ˈaɪərn /

性能指什么?

- 对于计算机来说，不同的角色所关注的**性能指标** (performance metric) 是不同的。
- 对于计算机系统的**用户**:
  - 关注用来处理程序所需要的时间
    - $t_{response} = t_{end} - t_{start}$
  - 又叫**响应时间 (response time)**
    - 考虑一下响应时间包括哪些?
- 对于计算机系统（数据中心）的**管理者**:
  - 关注每秒钟能完成的任务数量
  - 又叫**吞吐率 (throughput)**

CPU time

- 对于计算机架构师（设计者）：
  - 最关注的性能指标是**CPU time**: CPU真正用于**执行程序所花费的时间**
  - 当然，也需要关注能耗 (**energy, power**)
    - **能耗的重要性一直在增加**
- 在通常的认知里：一个量的 **值越大**，认为其**越好**。
  - 所以，将性能定义为：**性能 = 1/X**，期中X是执行时间
- 实际上，计算机处理一个程序的响应时间包括：

  CPU time + I/O time
- 我们后续重点讨论：CPU time

如何比较计算机的性能?

- 机器A的速度是机器B的速度的n倍，当且仅当:
  - **perf(A)/perf(B) = time(B)/time(A) = n**
- 机器A比机器B快 x%，当且仅当:
  - **perf(A)/perf(B) = time(B)/time(A) = 1 + x/100**

- 示例: time(A) = 10s, time(B) = 15s
  - **15/10 = 1.5 => A is 1.5 times faster than B**

  或者:

  - **15/10 = 1.5 => A is 50% faster than B**

问 题

- 基于你当前所掌握的专业知识，请回答：如何计算一个程序运行时所耗费的 CPU time?

- 通过将程序的处理分解为一条一条指令的处理来进行计算。
  - **Demo：CPU走马灯**

如何计算CPU time?

- 将程序的执行分解为指令的处理：
  - 硬件能够感知到的是指令的处理，而不是程序。
  - 程序是为了方便解决问题而引入的一种**抽象**；
- 在底层，一条指令的处理被CPU划分为**一个**或者**多个**时钟周期 (clock cycle)
  - 底层的状态机 (FSM) 在时钟的驱动下切换状态来完成指令在不同阶段的功能处理
- 那么，大家知道如何计算CPU time了吗?

内容概要

- 时间与性能
- Iron Law * – 时间
- 基准测试
- Amdahl’s Law

* Iron Law一词，来源于法律。 英 / ˈaɪən / 美 / ˈaɪərn /

评测性能的标准 — 时间

$$\text{处理器性能} = \frac{\text{Time}}{\text{Program}}$$

$$= \frac{\text{Instructions}}{\text{Program}} \times \frac{\text{Cycles}}{\text{Instruction}} \times \frac{\text{Time}}{\text{Cycle}}$$

(code size) (CPI) (cycle time)

CPU time的计算方法

影响CPU time的三要素

- Instructions/Program (程序运行对应的指令数)
  - 程序运行过程所**执行的指令数**，不是二进制代码里的指令数
  - 由 算法、编译器、ISA 决定
- Cycles/Instruction (指令处理所需时钟周期数)
  - 由ISA 和 CPU组织 决定
  - 比如**流水线**会降低这个指标
- Time/cycle (一个时钟周期所需要的的时间)
  - 由技术、组成、电路设计技术决定

优化CPU time的注意事项

- 要最小化**三要素的乘积**，而**不是某个独立的项。**
- 性能优化的常见错误是忽略了某个要素
  - 例如，通过改变ISA来减少指令的数量（第一个要素）
  - 然而，ISA的改变可能导致CPU实现变复杂，进而使得整个时钟周期（第三个要素）变长。
  - 换言之，该方法有可能得不偿失。
- 所以要谨记：三要素是相关的

$$\frac{\text{Instructions}}{\text{Program}} \times \frac{\text{Cycles}}{\text{Instruction}} \times \frac{\text{Time}}{\text{Cycle}}$$

(code size) (CPI) (cycle time)

性能分析示例A

1. 机器 A: clock 1ns, CPI 2.0, for program x
2. 机器 B: clock 2ns, CPI 1.2, for program x

问：哪个机器快，快多少?

答：假设X运行过程共有N条指令，那么：

Time/Program = instr/program x cycles/instr x sec/cycle

Time(A) = N x 2.0 x 1 = 2N

Time(B) = N x 1.2 x 2 = 2.4N

比较: Time(B)/Time(A) = 2.4N/2N = 1.2

所以，对于程序X来说，机器A比机器B快20%。

性能分析示例B

- 维持clock(A) @ 1ns，clock(B) @2ns，并假设CPI(B)=1.2，要获得相同的性能，CPI(A) 应该是多少?

A. 2　　B. 2.4　　C. 1.6　　D. 2.8

性能分析示例B

- 维持clock(A) @ 1ns，clock(B) @2ns，假设：CPI(B)=1.2，要获得相同的性能，CPI(A) 应该是多少?

答:

Time(B) / Time(A) = 1

即，(Nx2x1.2) / (Nx1xCPI(A)) = 1

=> CPI(A) = 2.4

内容概要

- 时间与性能
- Iron Law * – 时间
- 基准测试
- Amdahl’s Law

* Iron Law一词，来源于法律。 英 / ˈaɪən / 美 / ˈaɪərn /

如何综合衡量2台机器的性能?

- 运行某个程序，对比执行时间?
  - 这只能算部分正确
- 工程上用基准测试 (benchmark)
  - 选择**一组公认的程序**来评测性能
  - 通常会选择**一个参照机器**作为比较对象

如何综合衡量2台机器的性能?

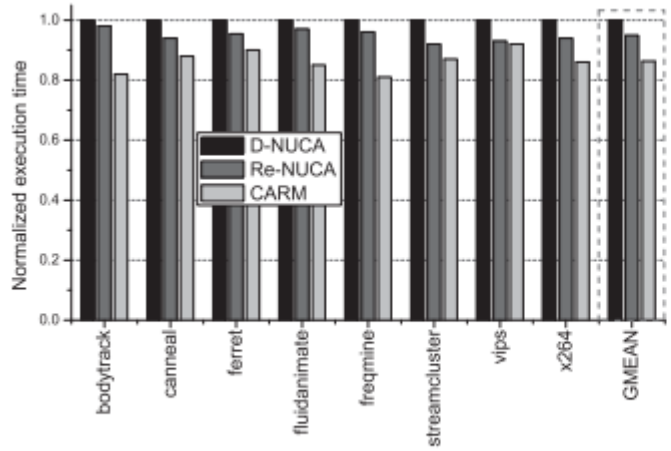


Fig. 10. Execution time normalized to D-NUCA.

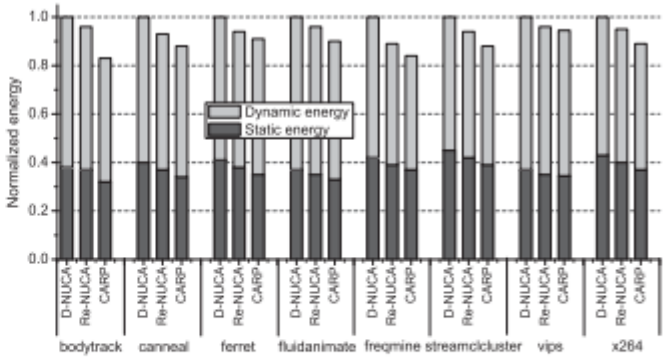


Fig. 11. Energy normalized to D-NUCA.

scheduling and CARM obtains the least reduction in execution time among the evaluated workloads.

- 基准测试有何好处?
  - 节省开销、消除争议
- 基准测试面临的问题有哪些?
  - 代表性? 诚实性? Benchmarketing?

感兴趣的同学，可以去调研不同领域的benchmark。

如何综合衡量2台机器的性能?

- 运行某个程序，对比执行时间?
  - 这只能算部分正确
- 工程上用基准测试 (benchmark)
  - 选择**一组公认的程序**来评测性能
  - 通常会选择**一个参照机器**作为比较对象
- 基准测试有何好处?
  - 节省开销、消除争议
- 基准测试面临的问题有哪些?
  - 代表性？诚实性？Benchmarketing?

感兴趣的同学，可以去调研不同领域的benchmark。

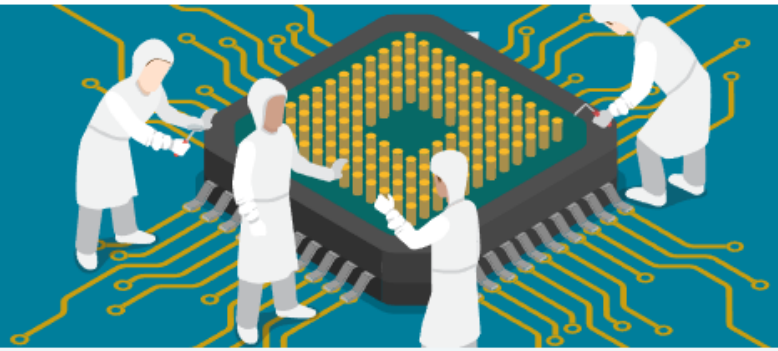
各类Benchmarks

Benchmarks | Services | Store | Support | About Us | Forums

CPU Benchmarks | Video Card Benchmarks | Hard Drive Benchmarks | Memory Benchmarks | PC Benchmarks | Software Marketshare | Database Benchmarks | Android Benchmarks | iOS Benchmarks

CPU Benchmarks

Over 1,000,000 CPUs Benchmarked



New Server CPU Performance

This chart comparing performance of CPUs designed for servers and workstation machines is made using thousands of PerformanceTest benchmark results and is updated daily.

Notes:

- While we try to keep this chart mainly desktop and laptop CPU free, there might be some previously mentioned processors in the list. This is because the classification of some CPUs span multiple classes.
- The chart shows only CPUs that had their benchmark first submitted within the last 24 months. Older server CPUs may not appear.

https://www.cpubenchmark.net/

TOP 500 List – June 2025

Rank	System	Cores	Rmax (PFlop/s)	Rpeak (PFlop/s)	Power (kW)
1	**El Capitan** - HPE Cray EX255a, AMD 4th Gen EPYC 24C 1.8GHz, AMD Instinct MI300A, Slingshot-11, TOSS, HPE DOE/NNSA/LLNL United States	11,039,616	1,742.00	2,746.38	29,581
2	**Frontier** - HPE Cray EX235a, AMD Optimized 3rd Generation EPYC 64C 2GHz, AMD Instinct MI250X, Slingshot-11, HPE Cray OS, HPE DOE/SC/Oak Ridge National Laboratory United States	9,066,176	1,353.00	2,055.72	24,607
3	**Aurora** - HPE Cray EX - Intel Exascale Compute Blade, Xeon CPU Max 9470 52C 2.4GHz, Intel Data Center GPU Max, Slingshot-11, Intel DOE/SC/Argonne National Laboratory United States	9,264,128	1,012.00	1,980.01	38,698
4	**JUPITER Booster** - BullSequana XH3000, GH Superchip 72C 3GHz, NVIDIA GH200 Superchip, Quad-Rail NVIDIA InfiniBand NDR200, RedHat Enterprise Linux, EVIDEN EuroHPC/FZJ Germany	4,801,344	793.40	930.00	13,088
5	**Eagle** - Microsoft NDv5, Xeon Platinum 8480C 48C 2GHz, NVIDIA H100, NVIDIA Infiniband NDR, Microsoft Azure Microsoft Azure United States	2,073,600	561.20	846.84	

https://top500.org/lists/top500/2025/06/

LINPACK Benchmark

TOP500 The List.

HOME | LISTS | STATISTICS | RESOURCES | ABOUT | MEDIA KIT

Home »About »The Linpack Benchmark

THE LINPACK BENCHMARK

As a yardstick of performance we are using the `best' performance as measured by the LINPACK Benchmark. LINPACK was chosen because it is widely used and performance numbers are available for almost all relevant systems.

The LINPACK Benchmark was introduced by Jack Dongarra. A detailed description and frequently asked questions can be found: http://www.netlib.org/utk/people/JackDongarra/faq-linpack.html

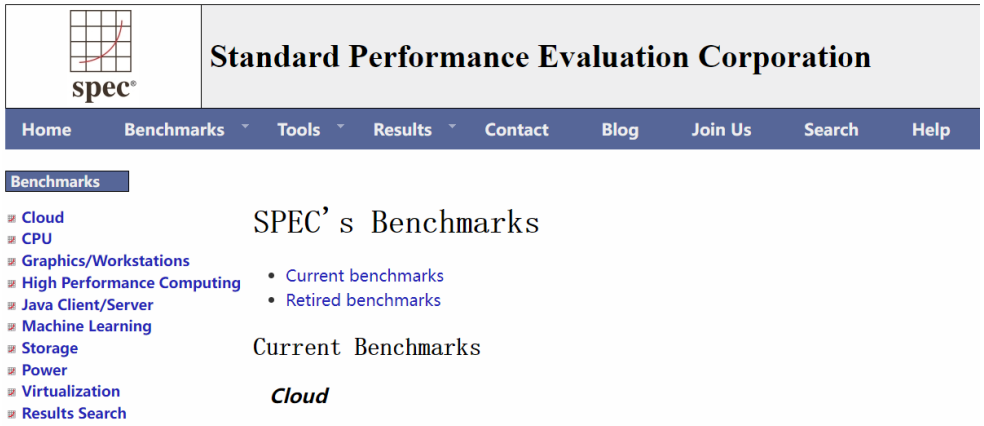
A parallel implementation of the Linpack benchmark and instructions on how to run it can be found at http://www.netlib.org/benchmark/hpl/.

The benchmark used in the LINPACK Benchmark is to solve a dense system of linear equations. For the TOP500, we used that version of the benchmark that allows the user to scale the size of the problem and to optimize the software in order to achieve the best performance for a given machine. This performance does not reflect the*overall performance* of a given system, as no single number ever can. It does, however, reflect the *performance of a dedicated system for solving a dense system of linear equations*. Since the problem is very regular, the performance achieved is quite high, and the performance numbers give a good correction of peak performance.



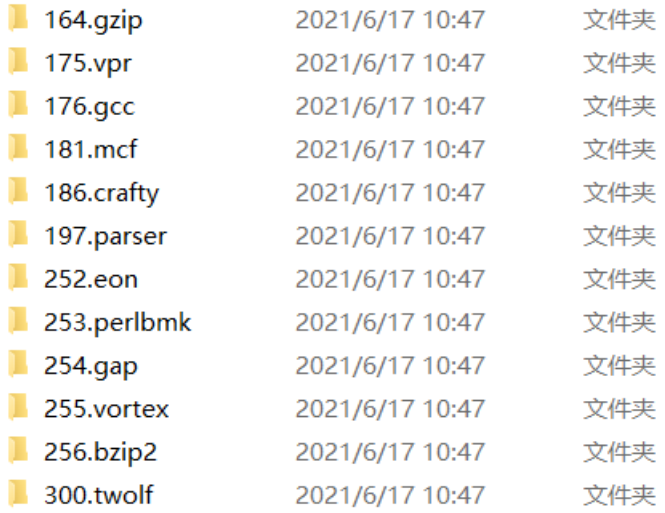
基准测试—以SPEC2000为例

- System Performance Evaluation Cooperative
  - Formed in 80s to combat benchmarketing
  - SPEC89, SPEC92, SPEC95, SPEC2000, SPEC2006
  - Latest one is SPEC2017
- 12 integer and 14 floating-point programs
  - Sun Ultra-5 300MHz as the **reference machine** has score of 100
  - Report GM of ratios to reference machine



基准测试—SPEC INT2000

Benchmark	Description
164.gzip	Compression
175.vpr	FPGA place and route
176.gcc	C compiler
181.mcf	Combinatorial optimiz
186.crafty	Chess
197.parser	Word processing, gran
252.eon	Visualization (ray traci..g)
253.perlbmk	PERL script execution
254.gap	Group theory interpreter
255.vortex	Object-oriented database
256.bzip2	Compression
300.twolf	Place and route simulator



Not toy programs, intensive computations or communications, etc.

基准测试—SPEC FP2000

Benchmark	Description
168.wupwise	Physics/Quantum Chrom
171.swim	Shallow water modeling
172.mgrid	Multi-grid solver: 3D pote
173.applu	Parabolic/elliptic PDE
177.mesa	3-D graphics library
178.galgel	Computational Fluid Dyna
179.art	Image Recognition/Neura
183.equake	Seismic Wave Propagatio
187.facerec	Image processing: face recognition
188.ammp	Computational chemistry
189.lucas	Number theory/primality testing
191.fma3d	Finite-element Crash Simulation
200.sixtrack	High energy nuclear physics accelerator design
301.apsi	Meteorology: Pollutant distribution



基准测试需要注意的陷阱

- Benchmark 没有代表性
  - 你是做数据库的，想测试I/O, 选SPEC是无用的
- Benchmark 太老
  - Benchmarks 很容易变老，从而没有代表性。
  - 由于Benchmarketing的压力，厂商会不断优化编译器、优化某些硬件部件、软件来获得高分。
  - 所以，benchmark需要周期性更新。

如何评测你的手机的性能?

安兔兔 benchmark



如何直观地表示性能?

- 基于下表中的数据回答：机器A快，还是机器B快?

	Machine A	Machine B
Program 1	1	10
Program 2	1000	100
Total	1001	110

- **问题抽象：**基准测试程序是1组workloads, 那么2台机器运行后有2组数据，如何确定那台性能好?
  - 算术均值、加权算术均值、
  - 调和均值 (**Harmonic Mean**)
  - 几何均值 (**Geometric Mean**)

用均值来表示性能结果

- 将机器A/B作为参照机器，将机器B/A的数据参照A/B处理

	Machine A	Machine B
Program 1	1	10
Program 2	1000	100
Total	1001	110

	Machine A	Machine B
Program 1	1	10
Program 2	1	0.1
Average	1	5.05

	Machine A	Machine B
Program 1	0.1	1
Program 2	10	1
Average	5.05	1

- 不可能同时成立
- 对于ratios，不能用算术均值。

可用几何均值

- 对于ratio，可用几何均值
- Geometric mean of ratios = $\sqrt[n]{\prod_{i=1}^{n} ratio(i)}$
- GM独立于参考机器
- 上面的例子中，无论参照机器A还是机器B，GM都是1。
- 此外， GM of ratios 与程序的执行时间不成比例。

- 有些同学将来会继续读研究生做学术研究，这部分内容对你有用。

内容概要

- 时间与性能
- Iron Law * – 时间
- 基准测试
- Amdahl’s Law

* Iron Law一词，来源于法律。 英 / ˈaɪən / 美 / ˈaɪərn /

Amdahl's Law

- 其动机是：**make the common case fast**
- **定义**：加速比 = old time/new time
- 假设把系统中占比 **f** 的部分加速 **s** 倍
  - new_time = (1-f) x old_time + (f/s) x old_time
  - speedup = old_time / new_time
  - speedup = old_time / ((1-f) x old_time + (f/s) x old_time)

$$= \frac{1}{1 - f + \frac{f}{s}}$$

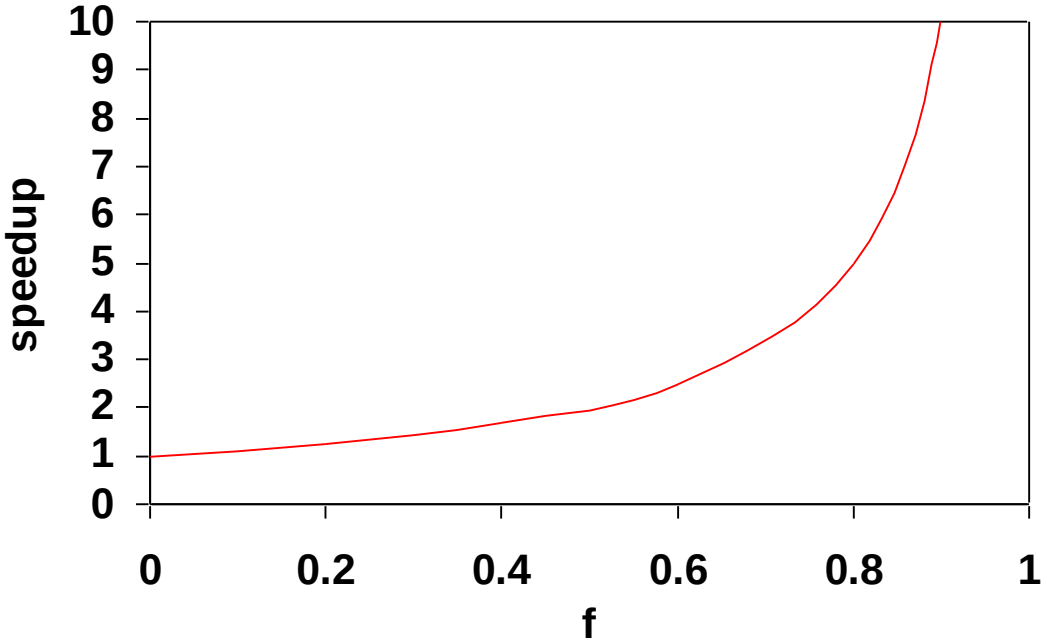
Amdahl's Law 示例

- Your boss asks you to improve performance by:
  - Improve the ALU used 95% of time by 10%
  - Improve memory pipeline used 5% of time by 10x

f	s	speedup
95%	1.10	1.094
5%	10	1.047
5%	∞	1.052

Amdahl's Law 的意义

- Make common case fast! (1 of 8 great ideas)



$$\lim_{s\to\infty} \frac{1}{1-f+\frac{f}{s}} = \frac{1}{1-f}$$

- 如果(1-f) 是显要的（不可忽略的）：
  - Speedup is limited!
  - 为什么1024核芯的GPU常见?
  - 为什么1024核芯的CPU却几乎看不到?

Limited parallelism

下一个主题是存储层次架构
请提前阅读下面的内容！

- Appendix B. Review of Memory Hierarchy
  - B.1 Introduction
    - Cache Performance Review
    - Four Memory Hierarchy Questions
    - An Example: The Opteron Data Cache
  - B.2 Cache Performance
    - Average Memory Access Time and Processor Performance
    - Miss Penalty and Out-of-Order Execution Processors
  - B.3 Six Basic Cache Optimizations
    - First Optimization: Larger Block Size to Reduce Miss Rate
    - Second Optimization: Larger Caches to Reduce Miss Rate
    - Third Optimization: Higher Associativity to Reduce Miss Rate
    - Fourth Optimization: Multilevel Caches to Reduce Miss Penalty
    - Fifth Optimization: Giving Priority to Read Misses over Writes to Reduce Miss Penalty
    - Sixth Optimization: Avoiding Address Translation during Indexing of the Cache to Reduce Hit Time
    - Summary of Basic Cache Optimization